Istruzioni: Scrivere immediatamente su ogni foglio cognome, nome e numero di matricola. È consentito utilizzare esclusivamente i fogli forniti dalla commissione. Tutto il materiale personale (quaderni, appunti, dispositivi elettronici), deve essere riposto nello zaino - che deve rimanere chiuso - o depositato sulla cattedra. I dispositivi elettronici (telefoni, smartwatch, fitness band, auricolari, tablet, ecc.) atti alla ricezione di informazioni devono essere spenti e non devono essere mai visibili durante la prova. La prova ha una durata di **3 ore**. Ogni violazione delle regole comporterà l'immediato annullamento della prova.

Esercizio 1

La Hero Association vi commissiona la realizzazione di un sistema informativo per gestire i propri iscritti. In particolare, Il sistema consentirà a un amministratore di registrare nuovi eroi, specificando il nome, l'alias, la classe di appartenenza (C, B, A o S), le abilità principali e altri dettagli rilevanti. Oltre alla registrazione, l'amministratore potrà assegnare nuove missioni a uno o più supereroi, definendo per ciascuna il tipo di minaccia, il livello di pericolo e l'area geografica interessata. Una volta avviata la missione, sarà possibile aggiornarne lo stato, indicando se è stata completata con successo oppure fallita.

Ogni supereroe avrà accesso al proprio profilo, dove potrà consultare l'elenco delle missioni assegnate e il proprio ranking, aggiornato in base alle performance e alla popolarità. I supereroi, inoltre, potranno richiedere di non fare più parte dell'associazione, inviando un'apposita richiesta. Gli amministratori possono accettare o rifiutare tale richiesta.

Il sistema prevede anche la possibilità di promuovere un supereroe che si sia distinto per le proprie gesta o abbia ottenuto particolare visibilità mediatica. In questi casi, l'amministratore seleziona l'eroe e ne modifica il ranking all'interno della classe o ne assegna una nuova classe di appartenenza. A seguito della promozione, il sistema pubblica automaticamente una notizia sui canali social ufficiali della Hero Association, utilizzando le API del servizio esterno OPSocial. Si precisa che tra una promozione e l'altra, per motivi burocratici, devono passare almeno 2 settimane. Se si prova a promuovere un eroe senza che sia passato sufficiente tempo dall'ultima promozione, il sistema deve mostrare un messaggio d'errore.

Si svolgano i seguenti punti, usando la propria conoscenza del dominio per derivare dettagli non definiti nei requisiti.

a. Modellare tutti i requisiti funzionali del sistema descritto sopra utilizzando uno Use Case Diagram;
b. Realizzare i mock-up dell'applicazione descritta, relativamente alla funzionalità di promozione di un eroe.
c. Dettagliare il caso d'uso relativo alla funzionalità di promozione di un eroe, per mezzo descrizioni testuali strutturate secondo un formalismo tabellare come quello proposto da A. Cockburn.

Esercizio 2

Un grande Ateneo italiano vuole commissionare la realizzazione di una nuova piattaforma di segreteria online che sfrutta l'intelligenza artificiale per offrire supporto agli studenti. Gli studenti potranno interagire con il sistema tramite una chat in linguaggio naturale, ed ottenere così informazioni su tasse, piani di studi, esami, etc. Il sistema, inoltre, potrà anche effettuare operazioni per conto degli utenti (e.g.: modifiche al piano di studi). Se lo desiderano, gli studenti potranno inoltre chiedere di interagire con un responsabile umano.

In questo contesto, vi è richiesto di contribuire alla redazione del documento di specifica dei requisiti software per la fornitura di cui sopra, fornendo almeno **tre requisiti funzionali** e **tre non funzionali** da inserire nel documento di specifica dei requisiti.

Esercizio 3

In un sistema informatico, la gestione delle utenze avviene secondo le modalità descritte di seguito. Un'utenza appena creata da un amministratore risulta inizialmente sospesa. L'utenza diventa attiva solo dopo che l'utente effettua il primo accesso utilizzando le credenziali fornite dall'amministratore e modifica contestualmente la password. In caso di fallimento di un'operazione di login, il sistema impone un blocco temporaneo di 60 secondi prima di consentire un nuovo tentativo, al fine di ridurre il rischio di attacchi brute force. Se l'utente fallisce 5 tentativi consecutivi di login, l'utenza viene bloccata per motivi di sicurezza. Un'utenza bloccata può essere sbloccata in due modi: tramite il codice PUK fornito insieme alle credenziali iniziali; tramite intervento diretto di un amministratore. Nel caso di intervento dell'amministratore, l'utenza torna allo stato di sospesa, e l'utente dovrà modificare nuovamente la password al primo accesso successivo. Se l'utente inserisce un codice PUK errato, l'utenza rimane bloccata fino a quando non interviene un amministratore.

Si specifichi tramite uno Statechart il comportamento di un'utenza, evidenziando i vari stati in cui un'utenza può trovarsi e le transizioni in funzione delle azioni di utenti e amministratore.

Esercizio 4

Il metodo statico *calcolaRimborso* della classe *TravelUtils* viene utilizzato per calcolare il rimborso da erogare ad un dipendente per un viaggio di lavoro. Il metodo prende in input quattro parametri:

- *kmPercorsi*: un float non negativo indicante la distanza (in km) dalla sede dell'azienda alla destinazione in cui si è svolto il viaggio di lavoro.
- *durata*: un intero non negativo che rappresenta la durata (in giorni) del viaggio di lavoro. I viaggi non possono avere durata superiore a 14 giorni.
- *isFestivo*: un booleano che indica se il viaggio copre o meno giorni festivi.
- *tipoDipendente*: una stringa che può assumere come valori: "DIRIGENTE", "FUNZIONARIO", "OPERATORE".

Il rimborso viene calcolato moltiplicando la durata del viaggio in giorni per un coefficiente determinato in base alla distanza e alla tipologia di dipendente, come da tabella seguente.

	$0 < kmPercorsi \leq 100$	$100 < kmPercorsi \leq 500$	$kmPercorsi > 500$
DIRIGENTE	100	150	200
FUNZIONARIO	75	100	150
OPERATORE	50	75	100

Se il viaggio copre anche giorni festivi, inoltre, viene sommato al rimborso calcolato come specificato sopra un ulteriore somma di € 200.

Se uno dei parametri non è valido, il metodo solleva una *ReimbursementException*.

a. Indicare, per ciascuno dei parametri del metodo *calcolaRimborso*, un partizionamento in classi di equivalenza per testing black-box;
b. Scrivere tre test JUnit con strategia Black Box per il metodo *calcolaRimborso*, indicando per ciascuno di essi quali classi di equivalenza copre. Si richiede inoltre che un test corrisponda a scenari in cui almeno uno dei parametri è non valido, e che i restanti due corrispondano a scenari in cui tutti i parametri sono validi. Ove possibile, i test devono coprire classi di equivalenza distinte.
c. Quanti test sono necessari per testare il metodo con strategia R-WECT? Motivare la risposta, descrivendo brevemente tale strategia.

Esercizio 2

Il metodo statico *calcolaImposta* della classe *TaxUtils* calcola le imposte dovute in un sistema a tassazione progressiva con due scaglioni. Il metodo prende in input quattro parametri:

- *imponibile*: un float non negativo indicante la base imponibile, in Euro.
- *soglia*: un float non negativo indicante la soglia di imponibile che determina il passaggio dal primo al secondo scaglione.
- *aliquota1*: un intero compreso tra 0 e 100, estremi esclusi. Indica l'aliquota di tassazione per il primo scaglione.
- *aliquota2*: un intero compreso tra 0 e 100, estremi esclusi. Indica l'aliquota di tassazione per il secondo scaglione.

Per esempio, se l'imponibile è 1000€, la soglia è fissata a 700€, e le aliquote sono rispettivamente al 10% e al 30%, l'imposta dovuta sarà calcolata come segue. Per la parte di imponibile rientrante nel primo scaglione, ovvero non superiore alla soglia, si applica l'aliquota del 10%, corrispondente a 70€. Per la parte di imponibile che rientra nel secondo scaglione, ovvero i restanti 300€, si applica la seconda aliquota al 30%, corrispondente a 100€ di imposta. In totale, le imposte nell'esempio ammontano a 70€ + 100€ = 170€.

Se uno dei parametri non è valido, il metodo solleva una *TaxException*.

d. Indicare, per ciascuno dei parametri del metodo *calcolaImposta*, un partizionamento in classi di equivalenza per testing black-box;
e. Scrivere tre test JUnit con strategia Black Box per il metodo *calcolaImposta*, indicando per ciascuno di essi quali classi di equivalenza copre. Si richiede inoltre che un test corrisponda a scenari in cui almeno uno dei parametri è non valido, e che i restanti due corrispondano a scenari in cui tutti i parametri sono validi. Ove possibile, i test devono coprire classi di equivalenza distinte.
f. Quanti test sono necessari per testare il metodo con strategia SECT? Motivare la risposta, descrivendo brevemente tale strategia.